SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

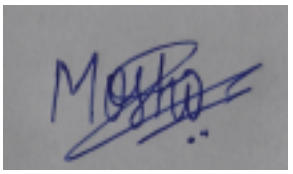
COURSE CODE: CSA09 COURSE NAME: Programming in Java COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 5 Total Marks: 100 ANNEXURE A **Coding Challenge**

Code of Conduct:

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm Handling	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	Code does not execute or gives incorrect results
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Unable to solve within time constraints
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Fails on multiple test cases
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	No clear algorithmic approach

ANNEXURE

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

ANSWER:

Main.java

Share Run

```java
import java.util.*;
class Book {
    private int id;
    private String title, author;
    private double price;
    private boolean issued = false;
    Book(int id, String title, String author, double price) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
    }
    int getId() {
        return id;
    }
    void issue() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        } else
            System.out.println("Already Issued");
    }
    void ret() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned");
        } else
            System.out.println("Already Available");
    }
    void display() {
        System.out.println(id + " " + title + " " + author + " " + price + " " +
                (issued ? "Issued" : "Available"));
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] b = new Book[10];
        int n = 0, ch;
        do {
            System.out.println("\n1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit");
            ch = sc.nextInt();
            switch (ch) {
                case 1:
                    System.out.print("Id Title Author Price : ");
                    b[n++] = new Book(sc.nextInt(), sc.next(), sc.next(), sc.nextDouble());
                    break;
                case 2:
                    for (int i = 0; i < n; i++)
                        b[i].display();
                    break;
                case 3:
                    System.out.print("Book Id : ");
                    int id = sc.nextInt();
                    for (int i = 0; i < n; i++)
                        if (b[i].getId() == id)
                            b[i].issue();
                    break;
                case 4:
                    System.out.print("Book Id : ");
                    id = sc.nextInt();
                    for (int i = 0; i < n; i++)
                        if (b[i].getId() == id)
                            b[i].ret();
                    break;
```

```
                case 5:
                    System.out.print("Book Id : ");
                    id = sc.nextInt();
                    for (int i = 0; i < n; i++)
                        if (b[i].getId() == id)
                            b[i].display();
                    break;
            }
        } while (ch != 6);
    }
}
```

Output

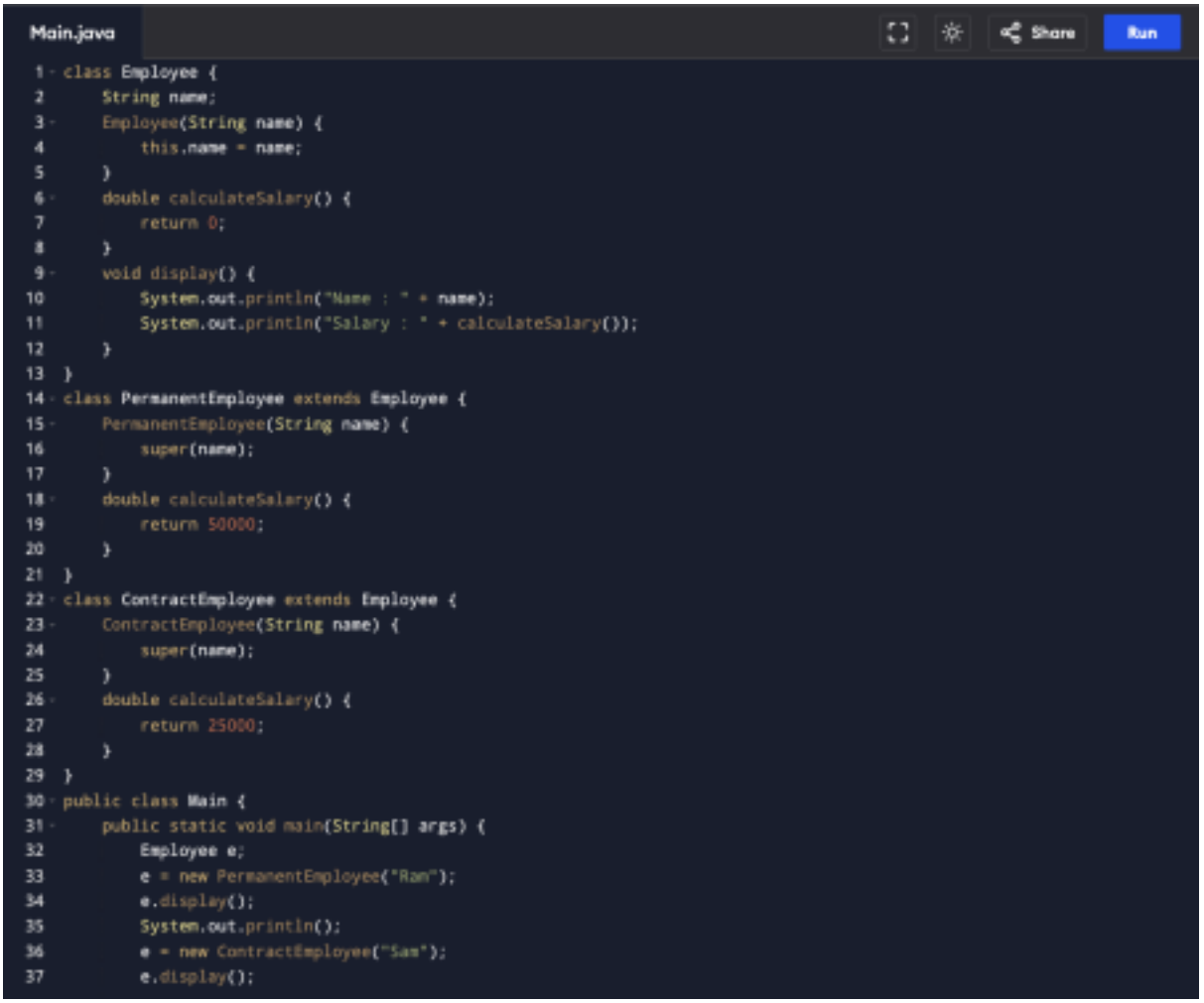
Clear

```
1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit
1
Id Title Author Price : 101 Java James 500

1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit
2
101 Java James 500.0 Available

1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit
3
Book Id : 101
Book Issued

1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit
4
Book Id : 101
Book Returned

1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit
1
Id Title Author Price : 102 Python John 600

1.Add 2.Display 3.Issue 4.Return 5.Search 6.Exit
5
Book Id : 102
102 Python John 600.0 Available
```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

ANSWER:



3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

ANSWER:

Main.java

Share Run

```java
class Vehicle {
    int calculateRent(int days) {
        return 0;
    }
}
class Car extends Vehicle {
    int calculateRent(int days) {
        return days * 1000;
    }
}
class Bike extends Vehicle {
    int calculateRent(int days) {
        return days * 300;
    }
}
class Bus extends Vehicle {
    int calculateRent(int days) {
        return days * 2000;
    }
}
public class Main {
    public static void main(String[] args) {
        Vehicle[] v = {
                new Car(),
                new Bike(),
                new Bus()
        };
        int days = 5;
        for (Vehicle x : v)
            System.out.println("Rent for " + days + " days = " + x.calculateRent(days));
    }
}
```

Output

Clear

```
Rent for 5 days = 5000
Rent for 5 days = 1500
Rent for 5 days = 10000

=== Code Execution Successful ===
```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

ANSWER:

Main.java

```java
class BankAccount {
    private int accountNumber;
    private String holderName;
    private double balance;
    BankAccount(int a, String h, double b) {
        accountNumber = a;
        holderName = h;
        balance = b;
    }
    void deposit(double amt) {
        balance += amt;
        System.out.println("Deposited: " + amt);
    }
    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            System.out.println("Withdrawn: " + amt);
        } else {
            System.out.println("Insufficient Balance");
        }
    }
    void display() {
        System.out.println("\nAccount No : " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance    : " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount b = new BankAccount(101, "Ram", 5000);
        b.deposit(2000);
        b.withdraw(1500);
        b.display();
    }
}
```

Output

```
Deposited: 2000.0
Withdrawn: 1500.0

Account No : 101
Holder Name: Ram
Balance    : 5500.0

=== Code Execution Successful ===
```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

ANSWER:

Main.java

```java
import java.util.*;
class Admission {
    void calculateFee() {
        System.out.println("UG Fee = 50000");
    }
    void calculateFee(int pg) {
        System.out.println("PG Fee = 80000");
    }
    void calculateFee(double scholarship) {
        System.out.println("Scholarship Fee = 20000");
    }
}
public class Main {
    public static void main(String[] args) {
        Admission a = new Admission();
        a.calculateFee();
        a.calculateFee(1);
        a.calculateFee(50.0);
    }
}
```

Output

```
UG Fee = 50000
PG Fee = 80000
Scholarship Fee = 20000

=== Code Execution Successful ===
```

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

ANSWER:

Main.java

```java
abstract class Shape {
    abstract void area();
}
class Circle extends Shape {
    void area() {
        double r = 5;
        System.out.println("Circle Area = " + (3.14 * r * r));
    }
}
class Rectangle extends Shape {
    void area() {
        int l = 4, b = 6;
        System.out.println("Rectangle Area = " + (l * b));
    }
}
class Triangle extends Shape {
    void area() {
        int b = 8, h = 5;
        System.out.println("Triangle Area = " + (0.5 * b * h));
    }
}
public class Main {
    public static void main(String[] args) {
        Shape[] s = {new Circle(), new Rectangle(), new Triangle()};
        for (Shape x : s)
            x.area();
    }
}
```

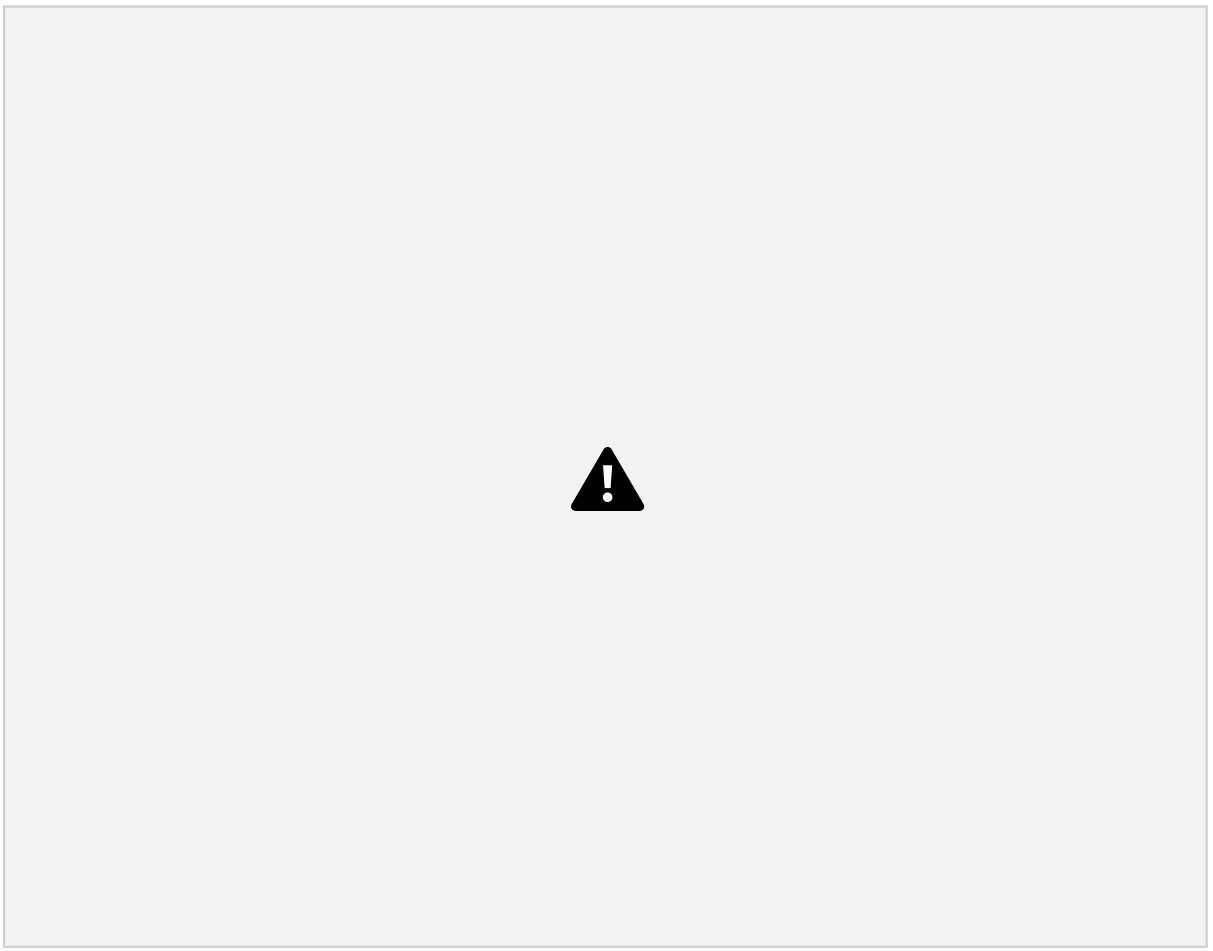
Output

```
Circle Area = 78.5
Rectangle Area = 24
Triangle Area = 20.0

=== Code Execution Successful ===
```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

ANSWER:



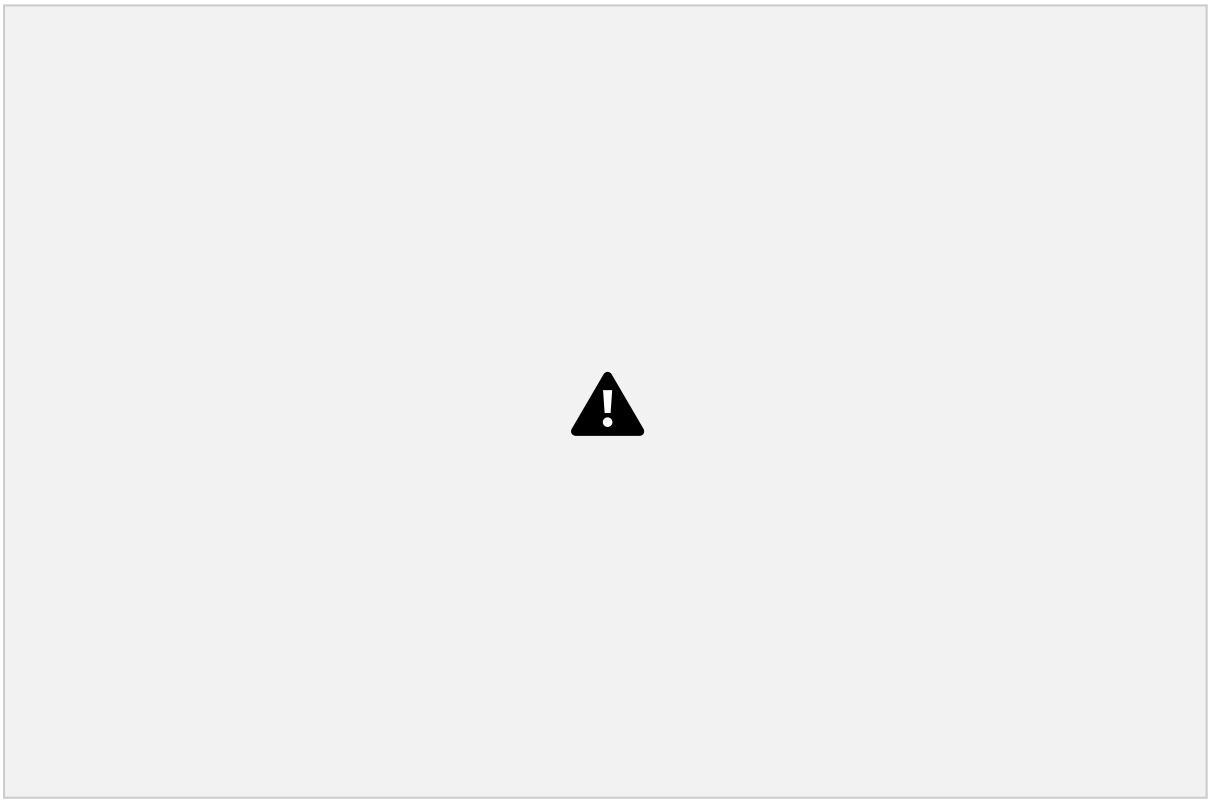
8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

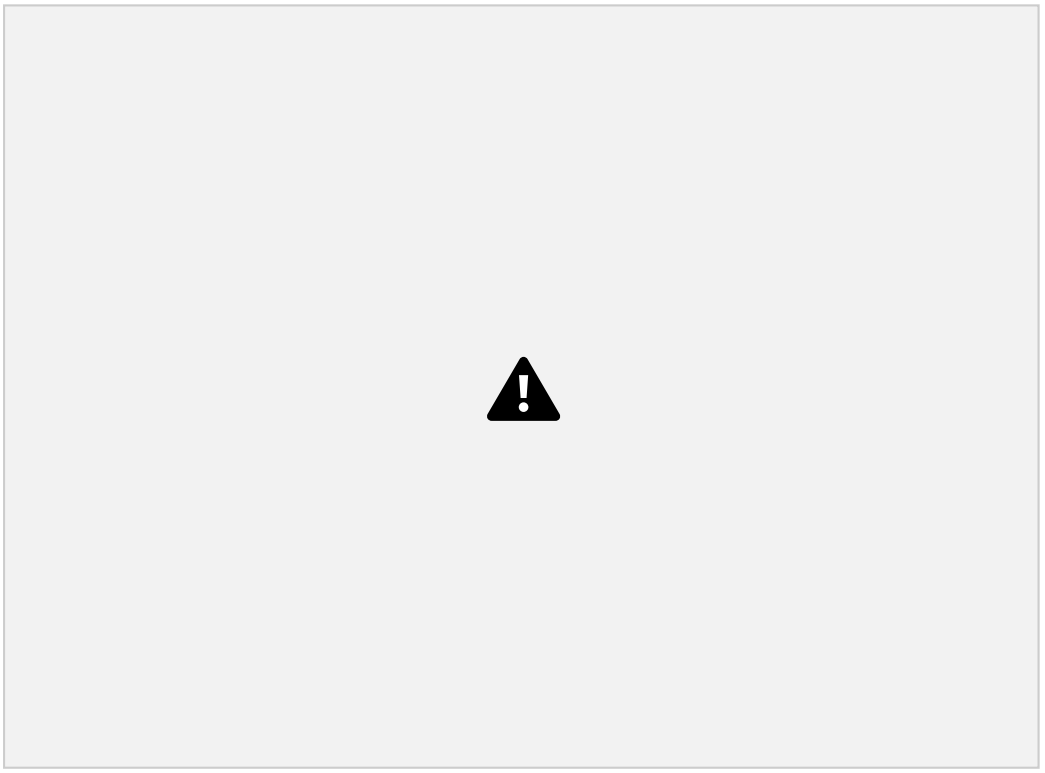
ANSWER:

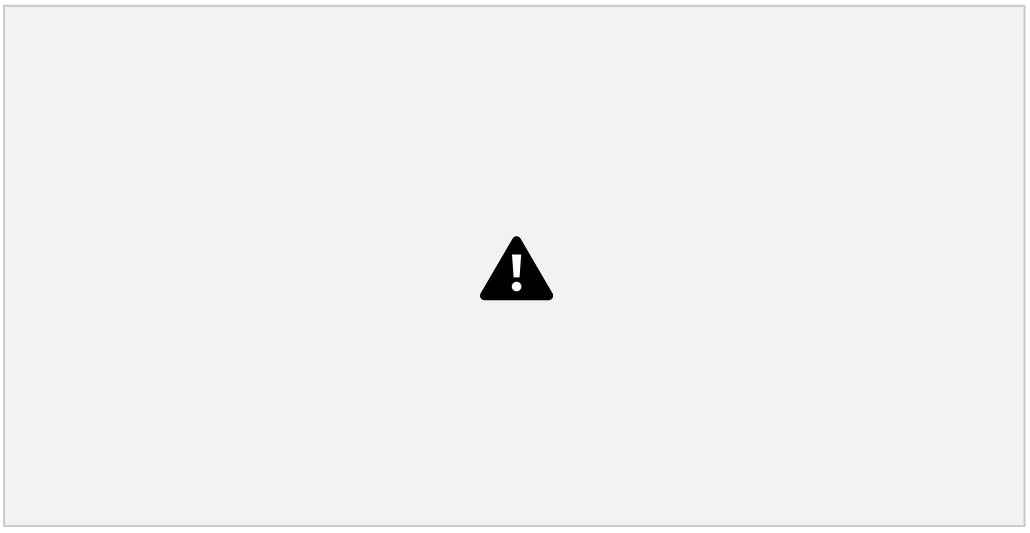


9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

ANSWER:





10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

ANSWER: